# Space Empire

## Supplément — Combat multi-joueurs & Déplacements

*Complément au document de conception technique v1*

Mars 2026

# 1. Relations entre joueurs et guildes

## 1.1 Statuts entre guildes

Trois statuts possibles entre deux guildes, tous réciproques :

| Statut | Déclenchement | Remarque |
|---|---|---|
| Neutre | Par défaut | Aucune relation établie |
| Alliés | Accord des deux guildes requis | Symétrique, nécessite validation mutuelle |
| Ennemis | Déclaration unilatérale suffit | Symétrique automatiquement |

## 1.2 Contrats d'alliance joueur ↔ joueur

- Un contrat d'alliance est un accord direct entre deux joueurs, indépendant des guildes.
- Le contrat est symétrique : si A est allié de B, B est allié de A.
- Impossible de signer un contrat avec un joueur d'une guilde ennemie à la sienne.
- Si la relation entre deux guildes devient « ennemis » après la signature d'un contrat entre deux de leurs membres, le contrat est automatiquement rompu.

> *Les alliances guilde↔guilde et joueur↔joueur coexistent mais ne se substituent pas. Un joueur peut être allié avec un joueur d'une guilde neutre via contrat direct.*

## 1.3 Impact sur le combat — résolution des camps

Deux joueurs sont considérés alliés dans un combat si :

- Ils ont un contrat d'alliance direct, OU
- Ils appartiennent à la même guilde, OU
- Leurs guildes respectives ont le statut « alliés ».

# 2. Système de combat multi-joueurs

## 2.1 Vue d'ensemble — les phases

```
T+0         Première flotte en statut ATTAQUE arrive sur la planète cible
            → Création d'un CombatEvent avec timer fixe (durée définie côté
serveur)
            → Notifications envoyées aux joueurs éligibles
            → Le déclencheur devient l'attaquant principal

T+[0..X]    Fenêtre de renfort
            → D'autres flottes peuvent rejoindre en ATTAQUE ou en DEFENSE
            → Chaque arrivée est horodatée (arrival_at)
            → Notifications de nouvelles arrivées envoyées au défenseur

T+X         Résolution du combat
            → Calcul des camps selon les relations entre joueurs
            → Simulation round par round avec arrivées échelonnées
            → Log généré, résultats persistés, Mercure notifie tous les
participants
```

> *Le timer démarre à l'arrivée de la PREMIÈRE flotte en statut attaque, peu importe qui l'a envoyée. Si le joueur A envoie sa flotte en premier mais que le joueur C (plus rapide ou plus proche) arrive avant lui, C déclenche le timer et A est traité comme un renfort attaquant.*

## 2.2 Résolution des camps

| Situation | Type | Résultat |
|---|---|---|
| A attaque B — seul | 1v1 | A vs B |
| A attaque B, C attaque B — A et C alliés | Coalition | A+C vs B |
| A attaque B, C attaque B — A et C non alliés | FFA | A vs B vs C |
| A attaque B, C défend B (allié ou non) | Défense extérieure | A vs B+C |
| FFA : défenses statiques de B | FFA | Défenses de B tirent sur A et C indifféremment |

### Règles du FFA

- Dans un FFA, chaque vaisseau cible indifféremment n'importe quel ennemi des deux autres camps.

- Les défenses statiques du défenseur tirent sur tous les attaquants (FFA ou non).
- Un défenseur extérieur (C défend B sans être allié) ne reçoit pas de dégâts des défenses de B.

## 2.3 Arrivées échelonnées

Les flottes qui rejoignent pendant le timer n'ont pas toutes le même moment d'entrée en combat. Cela est géré par les rounds :

- Les rounds sont calculés sur la durée totale du combat.
- Une flotte arrivant à T+X/2 entre au round le plus proche de T+X/2.
- Avant ce round, la flotte n'existe pas dans le combat — elle ne reçoit ni n'inflige de dégâts.
- Après ce round, elle participe normalement comme tous les autres vaisseaux.

> *Le calcul du combat est entièrement déterministe et réalisé côté serveur à T+X. Les arrivées échelonnées sont connues à l'avance (on connaît arrival_at de chaque flotte) et intégrées dans la simulation dès le début.*

## 2.4 Notifications

### Notification initiale — attaque détectée

Reçoivent la notification quand une planète est attaquée :

| Condition | Notifié ? | Remarque |
|---|---|---|
| Membre de la guilde du défenseur | ☑ Oui | |
| Joueur avec contrat d'alliance direct avec le défenseur | ☑ Oui | |
| Guilde alliée de la guilde du défenseur | ✗ Non | Alliance guilde ne suffit pas |
| Allié de l'attaquant | ✗ Non | L'attaque est discrète pour ses alliés |

### Identité de l'attaquant dans la notification

| Condition | Information affichée |
|---|---|
| Défenseur a découvert le système d'origine de l'attaquant | Nom du joueur attaquant visible |
| Défenseur n'a PAS découvert le système d'origine | Nom masqué — système d'origine visible uniquement |

### Notification de renfort entrant

- Quand une nouvelle flotte rejoint le combat (attaque ou défense) pendant le timer, le défenseur reçoit une notification.
- Les mêmes règles de visibilité sur l'identité s'appliquent (système d'origine découvert ou non).

## 2.5 Flottes en défense — cas particuliers

### Flotte de défense sans combat

- Si une flotte arrive en statut DEFENSE sur une planète et que le timer expire sans attaquant, la flotte repart automatiquement vers sa planète d'origine.
- Si une flotte part trop tard et arrive après T+X (combat déjà résolu), elle rentre directement — elle ne passe pas en orbite.

### Flotte en orbite chez un allié (hors contexte de combat)

- Statut distinct de DEFENSE : ORBITE.
- Une flotte en ORBITE chez un allié participe à la défense si une attaque survient pendant son stationnement.
- L'acceptation d'une flotte en orbite est manuelle par défaut.
- Exception : si un contrat d'alliance direct existe entre les deux joueurs, l'acceptation est automatique.
- La durée en orbite est limitée par le carburant (hydrogène) chargé dans la soute au départ.

## 2.6 Modèle de données — combat

```
CombatEvent
├── planet: Planet                        # planète cible
├── status: pending | in_progress | resolved
├── timer_start_at: datetime              # arrivée du 1er attaquant
├── resolve_at: datetime                  # timer_start_at + durée fixe
└── participants: CombatParticipant[]


CombatParticipant
├── combat_event: CombatEvent
├── fleet: Fleet
├── role: attacker | defender
├── arrival_at: datetime                  # pour les arrivées échelonnées
└── arrival_round: int | null             # calculé à T+X, round d'entrée en
combat ? est ce qu'on le garde ?
```

```
CombatLog                                  # généré après résolution
├── combat_event: CombatEvent
├── rounds: json                           # détail round par round (sérialisé)
├── outcome: attacker_wins | defender_wins | draw
└── created_at: datetime
```

# 3. Déplacements et carburant

## 3.1 Le carburant — hydrogène

L'hydrogène est la ressource utilisée comme carburant. C'est une mécanique générale qui s'applique à tous les déplacements de flotte.

Trois niveaux de consommation, définis par vaisseau dans le YAML :

| Situation | Clé YAML | Unité |
|---|---|---|
| Orbite / stationnement statique | fuel_orbit | H par heure |
| Déplacement intra-système stellaire | fuel_travel_system | H par unité de distance × vitesse_ratio |
| Déplacement inter-systèmes (galaxie) | fuel_travel_galaxy | H par unité de distance × vitesse_ratio |

## 3.2 Formules de consommation

### En déplacement

```
# La vitesse réduit la consommation proportionnellement

hydrogène_déplacement = consommation_base_vaisseau × vitesse_ratio × distance

# Exemple : flotte de 10 chasseurs, 100 ua à 100% de vitesse
# consommation_base_chasseur = 1 H/ua
# hydrogène = 10 × 1.0 × 100 = 100 H  — temps : 1h

# Même flotte à 50% de vitesse
# hydrogène = 10 × 0.5 × 100 = 50 H   — temps : 2h
```

### En orbite / stationnement

```
# Consommation fixe, indépendante de la vitesse

hydrogène_orbite = consommation_orbit_vaisseau × nb_vaisseaux × durée_heures
```

### Calcul total au départ (automatique)

```
hydrogène_total =
    hydrogène_déplacement_aller
```

```
  + hydrogène_orbite (timer combat OU durée orbite prévue)
  + hydrogène_déplacement_retour

# Ce montant est prélevé sur la planète d'origine au départ.
# Il occupe de la soute cargo.
# Le reste de la soute est disponible pour le fret.
```

## 3.3 La vitesse

- Le joueur choisit la vitesse entre 10% et 100% de la vitesse maximale de la flotte.
- La vitesse maximale de la flotte est limitée par le vaisseau le plus lent qui la compose.
- Réduire la vitesse réduit la consommation d'hydrogène en déplacement et augmente la durée du trajet de manière proportionnelle et linéaire.
- La consommation en orbite/stationnement n'est pas affectée par la vitesse de déplacement.

> *L'intérêt de réduire la vitesse est de libérer de la soute cargo (moins d'hydrogène prélevé pour le déplacement) afin d'emporter plus de fret. C'est un arbitrage tactique : arriver plus tôt ou emporter plus de ressources.*

## 3.4 La soute cargo

- Chaque vaisseau cargo dispose d'une capacité de soute définie dans son YAML (cargo_capacity).
- La soute totale de la flotte est la somme des soutes de tous les vaisseaux cargo présents.
- L'hydrogène du carburant et les ressources transportées partagent la même soute — ce n'est pas une réserve séparée.
- Le système refuse le départ si la soute totale est insuffisante pour l'hydrogène nécessaire au trajet.
- Une flotte sans vaisseau cargo ne peut pas effectuer de long trajet — pas de soute, pas d'hydrogène embarqué.

## 3.5 Types de mission et impact sur la soute

| Type de mission | Comportement de la soute |
|---|---|
| Orbite | Soute = hydrogène pour le trajet aller + durée de stationnement + retour |
| Transport | Soute = hydrogène + ressources à livrer. Le reste est disponible pour le retour. |
| Attaque | Soute = hydrogène + espace libre pour le butin récupéré après combat |

## 3.6 Perte de cargos en combat

- La panne sèche est impossible : le système calcule et réserve l'hydrogène nécessaire avant le départ.
- Si des vaisseaux cargo sont détruits pendant le combat, la soute disponible pour le retour est réduite.
- Les vaisseaux survivants rentrent quand même — ils consomment leur hydrogène de retour (déjà chargé).
- Le butin récupéré est réduit proportionnellement à la soute perdue. Les ressources excédentaires sont abandonnées sur place.

> *La perte de cargos en combat ne provoque pas de panne sèche car l'hydrogène de retour est déjà présent dans les soutes au moment du départ. En revanche, le butin récupérable est directement impacté par le nombre de cargos survivants.*

## 3.7 Statuts de flotte — récapitulatif complet

| Statut | Description |
|---|---|
| idle | Flotte stationnée sur sa planète d'origine, en attente d'ordre |
| traveling | En déplacement vers une destination |
| returning | En retour vers la planète d'origine |
| attacking | Arrivée sur la planète cible, en attente de résolution du combat (pendant le timer) |
| defending | En défense sur une planète alliée ou cible — repart si pas d'attaque à T+X |
| orbit | Stationnée en orbite chez un allié — durée limitée par le carburant embarqué |
| in_combat | Combat en cours (résolution en cours côté serveur) |

## 3.8 Modèle de données — mise à jour de Fleet

```
Fleet
├── owner: Player
├── home_planet: Planet
├── current_location: Planet | null       # null = en transit
├── destination: Planet | null
├── status: idle | traveling | returning
│          | attacking | defending | orbit | in_combat
├── speed_ratio: float                    # 0.10 à 1.00, choisi au départ
├── ships: FleetShip[]
├── departure_at: datetime | null
```

```
├── arrival_at: datetime | null
├── cargo: json # {'hydrogen': 200, 'metal': 0, ...}
├── hydrogen_reserved_fuel: float         # part réservée au carburant
└── cargo_available: float                # soute libre pour le fret

# hydrogen_loaded = hydrogen_reserved_fuel + fret_hydrogène_éventuel
# cargo_available = soute_totale_flotte - hydrogen_reserved_fuel
```

Départ {'hydrogen': 200}

Arrivée cible {'hydrogen': 100}

Après combat {'hydrogen': 100, 'metal': 50, 'carbon': 50}

Retour origine {'hydrogen': 0, 'metal': 50, 'carbon': 50}